



Spring Boot Interview Questions and Answers

By Ramesh Fadatare (Java Guides)

(Key points from Udemy Course: [Java Developer Interview Mastery: The Only Course You Need](#))

What Is Spring Boot and What Are Its Main Features?

- Spring Boot is a framework built on top of the Spring Framework.
- Its main purpose is to make Java application development easier, faster, and far more productive.
- Before Spring Boot, developers wrote a lot of configuration manually.
- They created XML files, registered beans, configured servers, and managed many setup steps themselves.
- Even small applications required too much effort and too much configuration.

What Spring Boot Solves

- Spring Boot removes heavy configuration.
- It provides sensible defaults.
- It auto-configures commonly used components.
- It helps developers build production-ready applications with minimal effort.
- This is why companies love Spring Boot, interviewers focus on it, and it has become the standard for modern backend development.

Interview Definition of Spring Boot

- Spring Boot is an opinionated framework built on top of Spring that simplifies Java development.
- It provides auto configuration, starter dependencies, and an embedded server.

- It allows developers to create standalone, production-ready applications quickly with minimal setup.

Opinionated Approach

- Spring Boot makes smart decisions for you.
- It chooses sensible defaults.
- It configures common things automatically.
- You don't worry about choosing the correct Jackson version.
- You don't manually configure the DispatcherServlet.
- You don't manually set up Hibernate or the server.
- Spring Boot handles all of that behind the scenes.

Auto Configuration

- Auto configuration is the heart of Spring Boot.
- It scans your classpath, checks your dependencies, and configures the components your project needs.
- With Spring Web: it sets up the dispatcher servlet, JSON handling, error handling, and the embedded Tomcat server automatically.
- With Spring Data JPA: it configures Hibernate, the entity manager, transaction management, and connection pooling.
- You start writing business logic immediately.

Starter Dependencies

- Spring Boot provides pre-packaged sets of libraries called starter dependencies.
- You do not have to search for compatible library versions manually.
- Starter Web gives everything needed for building REST APIs.

- Starter Data JPA gives everything needed for JPA.
- Starter Security gives everything for authentication and authorization.
- These starters eliminate version conflicts and make dependency management simple.

Embedded Server

- Spring Boot includes an embedded Tomcat, Jetty, or Netty server.
- Before Spring Boot, you deployed war files to external servers.
- With Spring Boot, the server is built in.
- When you run the application, the server starts automatically.
- No installation, no deployment, no manual configuration.
- This makes Spring Boot perfect for microservices because each service runs independently.

Spring Boot for Microservices

- Spring Boot is widely used to build microservices because each application is standalone and self-contained.
- When combined with Spring Cloud, it supports service discovery, API gateway routing, distributed configuration, load balancing, and fault tolerance.
- Together, Spring Boot and Spring Cloud form a complete microservices ecosystem.

Spring Boot Actuator

- Actuator provides production-ready endpoints for monitoring.
- It gives health checks, metrics, environment details, and system information.
- No extra code is required.
- Very useful for microservices, cloud deployments, and production monitoring.

DevTools

- Spring Boot improves development experience with DevTools.
- DevTools automatically restarts your application when you change code.
- It supports live reload for templates.
- This saves time and speeds up development.

Externalized Configuration

- Spring Boot supports externalized configuration using properties or YAML files.
- You can create multiple profiles: dev, test, staging, prod.
- The same application behaves differently based on the active profile.
- This avoids hardcoding values and keeps configuration clean.

Final Interview Answer

- Spring Boot is a modern, opinionated framework built on top of Spring that simplifies Java development.
- It removes boilerplate configuration and offers auto configuration, starter dependencies, and an embedded server.
- It helps developers build standalone, production-ready applications quickly.
- Spring Boot provides Actuator for monitoring, DevTools for rapid development, and external configuration support.
- Because it is lightweight, fast, and easy to deploy, it is widely used for microservices and cloud-native applications.
- It reduces complexity, increases productivity, and helps teams deliver scalable applications faster.

What is Spring Boot DevTools?

- Spring Boot DevTools is a development-time tool that improves the speed and smoothness of your development workflow.
- It is not used in production.

- It is not used for deployment.
- Its only purpose is to help developers work faster and get instant feedback while building applications.

Automatic Restart

- The main feature of DevTools is automatic restart.
- Whenever you change your Java code, templates, or resources, DevTools automatically restarts your Spring Boot application.
- You don't need to stop the app manually.
- You don't need to start it manually.
- DevTools watches for changes and restarts instantly, which saves a lot of time.

Live Reload Support

- DevTools provides live reload support.
- It detects changes to HTML files, CSS, and static resources.
- It reloads them automatically in your browser.
- You don't refresh the page yourself.
- Your UI updates instantly, which is very useful for Thymeleaf and front-end work.

Disabling Caching During Development

- Spring Boot normally caches templates for performance.
- Caching slows down development because changes don't appear immediately.
- DevTools disables caching automatically in development mode.
- This allows every change to appear instantly without extra configuration.

Faster Restarts Using Two Classloaders

- DevTools improves classpath handling by splitting the app into two classloaders.
- One classloader holds base libraries.
- The other classloader holds your project classes and resources.
- DevTools does not restart the entire application.
- It only reloads the parts that changed, making restarts extremely fast.

Environment-Based Behavior

- DevTools detects when the application is running in development mode.
- It automatically enables development-friendly settings.
- In production mode, DevTools is completely disabled.
- This ensures zero performance impact in production.

Interview-Friendly Explanation

- Spring Boot DevTools speeds up development by automatically restarting the application when code changes.
- It enables live reload for HTML and static resources.
- It disables caching so changes appear instantly.
- It is designed only for development and gives immediate feedback without manual restarts.

Practical Example

- When you update a controller method or modify a Thymeleaf template, DevTools instantly reloads the application or the page.
- You don't stop the server.
- You don't restart it manually.
- You simply save the file, and DevTools refreshes everything automatically.
- This lets you code faster and focus on building features.

Why DevTools Helps in Large Projects

- Manual restart time gets slower as Spring Boot projects grow.
- DevTools keeps the restart time fast and efficient.
- This greatly improves developer productivity.

What is Spring Boot Actuator?

- Spring Boot Actuator is a built-in module that provides production-ready monitoring and management features.
- It exposes useful endpoints to understand what is happening inside your application while it is running.
- Its goal is to give developers and DevOps complete visibility into the health, performance, and internal state of the application without writing any extra code.

What Actuator Provides

- Actuator provides endpoints for health checks, metrics, application information, environment properties, thread details, request statistics, logs, beans, and more.
- When enabled, your application becomes observable.
- This is essential in microservices and cloud environments.

Health Endpoint

- The health endpoint shows whether the application is up, down, or facing issues.
- It can check the health of databases, message brokers, caches, and external services.
- Provides instant feedback when something goes wrong.

Metrics Endpoint

- Shows CPU usage, memory consumption, JVM statistics, HTTP request counts, and response times.
- Helps diagnose performance issues, detect bottlenecks, and understand system behavior under load.

Environment Insights

- Shows active profiles, configuration properties, environment variables, and system settings.
- Eliminates the need to search logs or print values manually.

Beans Endpoint

- Lists all Spring Beans loaded in the application context.
- Helps debug and understand how auto configuration has wired the application.

Log-Level Management

- Allows dynamic log-level changes at runtime without restarting the application.
- Useful for debugging in production by increasing logs temporarily and switching back without downtime.

Integration with Monitoring Tools

- Actuator integrates with Prometheus, Grafana, Zipkin, Micrometer, and other monitoring platforms.
- Easy to build dashboards and visualize metrics and trends.
- Essential for microservices architectures.

Interview Explanation

- Actuator monitors, manages, and provides real-time insights into Spring Boot applications.
- Exposes endpoints like health, metrics, environment, beans, and logs.
- Improves observability by showing performance, resource usage, configuration, and system state without custom code.
- Essential for microservices, monitoring, debugging, and maintenance.

Practical Example

- After enabling Actuator, you can instantly check database health.
- Monitor how many requests the API has processed.
- View memory usage, thread counts, active profiles, and configuration details.
- Provides complete visibility while the app is running.

Security Note

- Actuator exposes only safe endpoints by default.
- Sensitive endpoints can be enabled manually and protected using Spring Security.

Which Embedded Servers Does Spring Boot Support, and How to Change the Default?

- Spring Boot supports multiple embedded servers, which means the server is built directly inside your application.
- You do not install anything separately or deploy a war file to an external server.
- You simply run your application and the server starts automatically.
- Embedded servers are a major reason Spring Boot apps are easy to run, package, and deploy.

Supported Embedded Servers

- Spring Boot supports three main embedded servers: Tomcat, Jetty, and Undertow.
- Tomcat is the default embedded server when using Spring Boot Starter Web.
- Jetty is lightweight and commonly used in high-performance environments.
- Undertow is known for its non-blocking architecture and is popular for reactive applications and lightweight microservices.

Changing the Default Server

- Spring Boot includes Tomcat by default through the Spring Boot Starter Web dependency.
- To change the default server, you remove the Tomcat dependency and add Jetty or Undertow instead.

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions>
    <!-- Exclude the Tomcat dependency -->
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<!-- Use Jetty instead -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jetty</artifactId>
</dependency>

```

- Spring Boot detects the new server on the classpath and configures it automatically.
- You never manually configure the server; changing dependencies is enough.

Interview-Ready Explanation

- Spring Boot supports Tomcat, Jetty, and Undertow as embedded servers.
- Tomcat is the default server.
- To switch servers, exclude Tomcat and add the Jetty or Undertow starter.
- Spring Boot automatically configures the new server without any manual setup.

Practical Example

- If you want to use Jetty instead of Tomcat, exclude Tomcat from the starter and add the Jetty starter.
- When you run the application, Jetty starts automatically as the embedded server.
- No extra configuration is needed because Spring Boot handles the switch.

What Does **@SpringBootApplication** Do?

- **@SpringBootApplication** is a very important Spring Boot annotation.
- It looks simple, but it actually activates three major Spring Boot features behind the scenes.
- Most Spring Boot developers want auto-configuration, component scanning, and the ability to add extra configuration.
- **@SpringBootApplication** brings all three features together in a single, convenient annotation.

What **@SpringBootApplication** Enables

- **@SpringBootApplication** activates three other core annotations under the hood.
- The first is **@EnableAutoConfiguration**, which triggers Spring Boot's auto-configuration mechanism.
- It tells Spring Boot to look at the libraries on the classpath and automatically configure beans like Tomcat, Jackson, Hibernate, DataSource, and Spring MVC.
- The second annotation is **@ComponentScan**, which scans the package of the main class.
- It automatically detects **@Component**, **@Service**, **@Repository**, and **@Controller** without manual registration.

- The third annotation is **@Configuration**, which allows the class to define beans using **@Bean** methods and import other configuration classes.

@SpringBootApplication is a shortcut that combines **@Configuration**, **@EnableAutoConfiguration**, and **@ComponentScan**.

Interview Answer & Practical Example

- **@SpringBootApplication** is a combination of **@Configuration**, **@EnableAutoConfiguration**, and **@ComponentScan**.
- It enables auto-configuration, scans the current package for Spring components, and allows the application class to register additional beans.
- It is the primary annotation that bootstraps every Spring Boot application.
- When you create a new Spring Boot project, you only use **@SpringBootApplication**.
- You don't add XML, you don't register controllers, and you don't configure the server.
- You run the application and Spring Boot sets up everything automatically—controllers are discovered, beans are created, and the server starts.

Difference Between Spring Framework and Spring Boot

Category	Spring Framework	Spring Boot
Primary Purpose	A comprehensive, low-level framework for building enterprise applications. Provides DI, AOP, transactions, templates, and infrastructure tools.	Built on top of Spring. Its mission is to simplify Spring development using auto-configuration, defaults, and runtime simplicity.
Configuration Style	Heavy manual configuration. Requires XML, bean definitions, view resolvers, and multiple setup steps.	Auto-configuration + opinionated defaults. Configures most components automatically based on dependencies.
Project Setup	Requires adding many dependencies manually and ensuring version compatibility. Initial setup takes time.	Uses Spring Initializr to generate a complete project instantly with selected dependencies.
Web Server	Depends on external servers like Tomcat or Jetty. WAR deployment is common.	Comes with embedded Tomcat, Jetty, or Undertow. Server runs automatically when you start the application.
Deployment	Typically produces WAR/EAR files deployed to an external server.	Produces standalone JAR files with an embedded server. Run with a single

command—no external server needed.

Microservices Support

Manual setup required for discovery, routing, config, and tracing.

Seamless integration with Spring Cloud. Provides built-in discovery, gateway, config server, load balancing, and tracing.

Production-Ready Features

Requires extra configuration or third-party tools for monitoring and health checks.

Includes Actuator out of the box: health, metrics, environment, logs, and monitoring endpoints.

Properties & Environment Config

Requires manual property loading and source management.

Provides unified configuration using properties/YAML with built-in support for environment profiles (dev, test, prod).

Testing

Requires more setup for mocks, contexts, and integration testing.

Provides auto-configured test environments with `@SpringBootTest`, `@MockBean`, and simplified testing.

Difference Between @RequestParam and @PathVariable

Aspect	@RequestParam	@PathVariable
What It Does	Extracts data from query parameters in the URL. Example: <code>/search?keyword=laptop</code>	Extracts data from the URL path itself. Example: <code>/products/42</code> where <code>42</code> is the path variable
Where in the URL	Appears after the ? in the URL, often as key-value pairs. E.g., <code>/users?country=India&age=25</code>	Appears directly inside the URL path . E.g., <code>/users/25</code>
Example URL	<code>/user?id=101</code> — more flexible but slightly verbose	<code>/user/101</code> — more clean and direct , ideal for resource access
When to Use	Use when values are optional or used for filtering/searching/pagination . Good for flexible data input	Use when values are required and identify a specific resource like a user or product
Optional vs Required	Values can be optional . You can provide default values or skip them entirely	Values are typically required . The URL won't work without the specified value
Semantics in REST Design	Represents how you want the resource — used to customize behavior (filters, sorting)	Represents what resource you want — used to locate a specific item by its identifier

Difference Between @Component and @Bean

Aspect	@Component	@Bean
Definition	Class-level annotation. Marks a class as a Spring-managed component . Discovered automatically during component scanning.	Method-level annotation. Placed inside a class annotated with <code>@Configuration</code> . Registers the returned object as a bean .
How They Work	Spring scans for <code>@Component</code> , <code>@Service</code> , <code>@Repository</code> , and <code>@Controller</code> . It automatically creates instances and registers them in the context.	<code>@Bean</code> is manually declared inside a configuration class. Spring runs the method and adds the returned object to the application context.
Use Case	Ideal for your own classes like services, repositories, and controllers that Spring should discover and manage automatically .	Use when you need full control over bean creation — like registering third-party classes or applying custom logic during setup.
Flexibility	Spring handles instantiation — uses default constructor or constructor injection .	Offers full control — you can write logic , inject other beans as method parameters , and return custom objects .
Configuration Location	Goes directly on the class you want to be managed.	Must be placed inside a class annotated with <code>@Configuration</code> , which acts like a blueprint for creating beans.
Execution & Lifecycle	Results in a Spring-managed bean. Supports lifecycle callbacks like <code>@PostConstruct</code> , dependency injection, and default scope behavior.	Also results in a managed bean. Allows custom setup logic inside the method , offering explicit lifecycle behavior .

Difference Between Spring Boot and Microservices

Topic	Spring Boot	Microservices
Definition	A framework from the Spring ecosystem . Used to build backend applications quickly and efficiently with less boilerplate code .	An architectural style . Describes how to structure an app by splitting it into small, independently deployable services .
Purpose	Goal is to simplify Java backend development . Handles auto-configuration, dependency management , and builds production-ready apps faster.	Goal is to organize large systems into small, focused services . Each handles one business function and communicates over the network .
Scope	Focused on building a single app or service — works for monoliths or microservices .	Describes the entire system made of multiple independent apps , possibly built in Spring Boot, Node.js, Go , etc.
Relationship	Often used to build microservices. It's lightweight, fast, and easy to deploy , making it ideal for service development.	Microservices architecture often uses Spring Boot to implement each service. Spring Boot is the engine; microservices is the design .
Example	A single REST API with login, registration, profile — all inside one Spring Boot app.	Split into User Service, Auth Service, Notification Service — each deployed separately , communicating over HTTP.
Deployment	Deployed as a single .jar or .war file — just like a regular Java app.	Deployed as multiple containers (e.g., Docker). Each service runs, scales, and stores data separately .
Technology vs Design	A technology choice — focuses on how you code, run, and package an app.	An architectural decision — focuses on how you organize, split, and manage a large system of apps.

Difference Between Spring Boot and Spring Cloud

Category	Spring Boot	Spring Cloud
Definition	A framework that simplifies building Java applications. Enables quick setup , REST APIs, embedded servers like Tomcat — with minimal config .	A set of tools built on top of Spring Boot to build and manage distributed systems — like microservices. Handles config, discovery, resilience, etc.
Purpose	Focused on simplifying backend development . Handles dependency injection , logging, validation, and lets you focus on business logic .	Designed to support microservices architecture . Helps services talk to each other , manage settings, and stay resilient in cloud environments.
Use Case	Ideal for building standalone apps : REST APIs, batch jobs, monoliths. Lets you run with zero configuration .	Best when managing multiple Spring Boot apps . Useful for centralized config , circuit breakers , service discovery , and tracing .
Core Features	Offers auto-configuration , starter dependencies , actuator endpoints , and embedded servers . Simplifies building one app .	Provides tools like Eureka (service discovery) , Config Server , Zuul/Gateway , Hystrix . Focuses on how multiple apps work together .
Independence	Can be used alone . Doesn't require Spring Cloud. Great for monoliths or small backend apps.	Depends on Spring Boot . Adds distributed features on top . You start with Spring Boot , then layer Spring Cloud for scaling.
Complexity	Beginner-friendly . You can get an app running in minutes with starters and annotations.	Advanced . Involves service registries , gateways , config servers — meant for experienced teams working at scale.

Difference Between Spring Security and OAuth2

Aspect	Spring Security	OAuth2
What They Are	A security framework in the Spring ecosystem. Handles authentication, authorization, login, logout, CSRF protection, and password encoding.	A protocol, not a framework. An open standard for delegated access — used by platforms like Google, Facebook, GitHub to share data securely.
Role in Application	Provides infrastructure for handling security inside your app — decides who can log in, what resources they can access, and how sessions are managed.	Used for secure token exchange between apps and authorization servers. Lets users log in with third-party accounts (e.g., Sign in with Google).
Real-World Usage	Used when you secure your app directly — form logins, role-based access, and JWT validation.	Used when you delegate login to an external provider — authenticate users and retrieve profile info without handling passwords.
Token Handling	Handles sessions, cookies, or JWT tokens. Manages token validation, role checks, and logout.	Defines access tokens, refresh tokens, and authorization grants. Tokens are issued by auth servers and passed between services.
Integration Together	Has built-in support for OAuth2 via Spring Security OAuth and Spring Authorization Server. Enables SSO, delegated access, and provider logins.	Needs a framework like Spring Security to be implemented. OAuth2 alone cannot secure Java apps — it's just the protocol definition.
Learning Curve	Powerful but complex. Requires understanding filters, interceptors, annotations. Gives fine-grained control once configured.	Adds new concepts like clients, scopes, token types, and grant types (e.g., Authorization Code, Client Credentials). Takes time to master.

Difference Between @Controller and @RestController

Aspect	@Controller	@RestController
Purpose	Used in Spring MVC applications where the goal is to return web views like <code>.html</code> , <code>.jsp</code> , or Thymeleaf templates.	Used to build RESTful web services where the response is typically data, such as JSON or XML, not a view/template.
Response Handling	Returns a view name. If you return a string like <code>"home"</code> , Spring resolves it to a file like <code>home.html</code> or <code>home.jsp</code> .	Returns the string or object directly in the HTTP response body. <code>"Hello"</code> is returned as plain text or serialized JSON/XML.
Underlying Annotations	Purely <code>@Controller</code> , without automatic JSON conversion.	Combination of <code>@Controller</code> and <code>@ResponseBody</code> . The <code>@ResponseBody</code> tells Spring to serialize the return value.
When to Use	Best suited for server-side rendered web pages like dashboards, admin panels, or apps using JSP, Thymeleaf, or Freemarker.	Best for REST APIs, microservices, or frontend-backend communication (e.g., React, Angular, mobile apps).
Return Behavior Example	<code>return "dashboard"</code> looks for and renders a template file named <code>dashboard.html</code> .	<code>return "dashboard"</code> sends the literal text <code>"dashboard"</code> in the HTTP response body.
Integration & Flexibility	Commonly used for rendering HTML UI. Can be combined with other MVC features like <code>ModelAndView</code> .	Can coexist with <code>@Controller</code> in the same project. Used exclusively for API endpoints and structured data exchange.